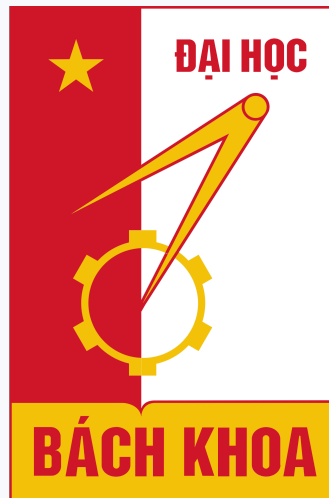


HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATIONS
TECHNOLOGY

oOo



REPORT

CAPSTONE PROJECT

TOPIC 3 : Tree Data Structure Visualization Application

Course : Object-Oriented Programming

Group : 14

Members : Nguyen Tien Dung – 202417116
: Nguyen Khac Khiem – 202417142
: Tran Quang Thang – 202417196
: Vu Khanh Toan – 202417204
: Nguyen Xuan Thuy – 202417202

Supervisor : Dr.Nguyen Thi Thu Trang

Hanoi – June 2026

Abstract

This report presents the design and implementation of **Tree Operation**, a JavaFX desktop application for visualizing tree data structure operations. The project supports several tree types, including General Tree, Binary Tree, Binary Search Tree, AVL Tree and Red-Black Tree. The application helps users understand tree algorithms through step-by-step visualization, highlighted nodes, algorithm messages and animation playback. The system is designed using object-oriented programming principles and follows the MVC architecture.

Contents

1	Project Introduction	3
1.1	Problem Statement	3
1.2	Background Knowledge	3
1.3	Proposed Approach	4
1.4	System Architecture	4
1.5	Intended Users	4
2	Team Member Contributions	5
3	Technical Foundation	5
3.1	Technology Stack	5
3.2	User Interface Framework and Screens	6
3.3	Supported Tree Algorithms	6
4	Detailed Design	7
4.1	Use Case Diagram	7
4.1.1	System Actor	7
4.1.2	General Functions	7
4.2	Class Diagrams	8
4.2.1	General Class Diagram	8
4.2.2	Detailed Class Diagram	10
4.3	Important Runtime Flow	15
4.4	Operation Step Model	15
4.5	Visualization and Animation Design	16
4.6	Layout Design	16
5	Object-Oriented Design and Techniques	16
5.1	Encapsulation	17
5.2	Inheritance	17
5.3	Polymorphism	18
5.4	Abstraction	19
5.5	Association, Aggregation and Composition	20
5.5.1	Association	20
5.5.2	Aggregation	20
5.5.3	Composition	20
5.6	Design Patterns	21

5.6.1	Model-View-Controller Pattern	21
5.6.2	Factory Pattern	21
5.6.3	Strategy Pattern	21
5.6.4	Singleton Pattern	22
5.7	Exception Handling	22
6	User Guide	23
6.1	System Requirements	23
6.2	Installation	23
6.2.1	Building from Source	23
6.2.2	Project Entry Point	23
6.3	Using the Application	23
6.3.1	Initialization	23
6.3.2	Executing Operations	24
6.3.3	Animation Control	25
6.3.4	Visual Aids	25
6.4	Video Demonstration	26
7	Conclusion	26
7.1	Key Achievements	26
7.2	Current Limitations	26
7.3	Future Enhancements	27
7.4	Learning Outcomes	27

1 Project Introduction

1.1 Problem Statement

Data structures and algorithms are fundamental topics in computer science. Tree data structures are especially important because they appear in indexing, searching, parsing, file systems, artificial intelligence and many other areas. However, many students find tree algorithms difficult to understand from static diagrams alone.

The difficulty becomes more visible when the algorithm changes the shape of the tree. Operations such as insertion, deletion, traversal, AVL rotations, Red-Black recoloring and Red-Black fix-up require several intermediate states. If users only see the final result, they cannot easily follow why the algorithm moves to a branch, replaces a node, updates height, rotates a subtree or changes a node color.

This project addresses that learning problem by building a JavaFX desktop application that visualizes tree operations step by step. The application lets users create and modify several tree types while observing the algorithm messages, highlighted steps and animated changes on a canvas.

1.2 Background Knowledge

This project is built around several tree data structures. A tree is a hierarchical structure consisting of nodes connected by parent-child relationships. The project defines a common abstract class, **AbstractTree**, for all tree types. It stores the root node, supports a listener for algorithm steps, and declares common operations such as create, insert, delete, update, traverse, search, height calculation and node counting. This common interface allows the application to process different tree types through the same set of operations while each concrete tree still keeps its own algorithmic rules.

A **General Tree** allows each node to have any number of children. In this project, insertion requires an existing parent value, then the new child is attached to that parent. Searching is performed by visiting the current node and recursively scanning its children. Deletion removes the target node from its parent; if the root is deleted, the whole tree becomes empty. The General Tree supports pre-order traversal, post-order traversal and breadth-first traversal. In-order traversal is not used because it is not naturally defined for a general tree with an arbitrary number of children.

A **Binary Tree** restricts each node to at most two children: a left child and a right child. Unlike a binary search tree, this tree does not use value ordering to decide where a node should be placed. Instead, insertion depends on the selected parent node and fills the left child before the right child. Searching must scan both branches because there is no smaller-left and larger-right rule. The tree supports in-order, pre-order, post-order and breadth-first traversal.

A **Binary Search Tree** adds an ordering rule to the binary tree structure. For every node, smaller values are stored in the left subtree and larger values are stored in the right subtree. During insertion or search, the algorithm compares the target value with the current node and then moves left or right. Deletion follows the standard BST cases: deleting a leaf node, deleting a node with one child, or replacing a node with its in-order successor when it has two children. This structure makes the algorithm easier to follow because each comparison determines a clear direction.

An **AVL Tree** is a self-balancing binary search tree. It performs insertion and deletion similarly to a BST, but after each structural change it updates node heights and checks the balance factor. The balance factor is calculated from the height difference between the left and right subtrees. If the tree becomes left-heavy or right-heavy beyond the allowed range, rotations are used to restore balance. The project handles left rotation, right rotation and double-rotation cases through the rebalance process.

A **Red-Black Tree** is another self-balancing binary search tree. Instead of storing heights, each node has a color, either red or black. A new non-root node is first inserted as a red node using normal BST ordering, then the tree performs a fix-up process to preserve red-black rules. The fix-up may recolor the parent, uncle and grandparent, or perform left and right rotations. The root is always recolored black after insertion if necessary. During deletion, the tree also keeps track of the removed node color and applies a delete fix-up when a black node removal may violate the tree properties.

1.3 Proposed Approach

The application is named **Tree Operation**. It provides an interactive workspace where users can select a tree type, input integer values and execute operations such as create, insert, delete, update, search and traverse.

The solution has four main parts:

- **Tree model layer:** implements the logical behavior of General Tree, Binary Tree, Binary Search Tree, AVL Tree and Red-Black Tree.
- **Step listener layer:** standardizes algorithm events with `StepType`, `AnimationStep` and `TreeOperationListener`.
- **Visualization layer:** maps logical nodes to visual nodes, calculates layout and renders nodes and edges on JavaFX Canvas.
- **Controller layer:** connects user actions to model operations, records history, updates statistics and plays animation sequences.

The most important design idea is that a tree model does not draw anything directly. Instead, the model emits semantic steps such as `COMPARE`, `GO_LEFT`, `INSERT_NODE`, `ROTATE_LEFT` or `RECOLOR`. The visualization controller listens to these steps and chooses the proper animation strategy.

1.4 System Architecture

The project follows the Model-View-Controller architecture.

- **Model:** packages `com.model.tree`, `com.model.node`, `com.model.step`, `com.model.history` and `com.model.exception`. These classes store data, implement algorithms and emit operation steps.
- **View:** FXML files, CSS files and visualization view models under `com.view`. The view defines the main menu, tree selection screen, workspace, settings page, credit/help page and canvas drawing models.
- **Controller:** packages under `com.controller`. Controllers process button clicks, navigation, tree selection, workspace operations and animation playback.

1.5 Intended Users

The application targets three main groups of users:

- **Students:** The application helps students understand tree data structures more easily through step-by-step visualization of insertion, deletion, searching and traversal operations.

- **Teaching assistants and lecturers:** The application can be used as a visual teaching tool in lectures or lab sessions, making tree algorithms easier to explain than using static diagrams only.
- **Developers:** Developers can use the application to test, debug and compare the behavior of different tree structures such as BST, AVL Tree and Red-Black Tree.

2 Team Member Contributions

The following table summarizes the main working areas of Group 14 according to the implemented project modules.

Member	Student ID	Contribution
Nguyen Tien Dung	202417116	Controller development, view designing and building, bug fixing, refactoring, navigation flow implementation, design class diagram, and use case diagram support.
Nguyen Khac Khiem	202417142	Developed controller logic, designed and built view components, supported animation handling, implemented navigation flow, refactored code, and contributed to class diagram and use case diagram design.
Tran Quang Thang	202417196	Model architecture design, tree data structure algorithm implementation, pseudo-code display implementation, controller connection support, report writing support, and use case diagram design.
Vu Khanh Toan	202417204	Model diagram design, BST, AVL and Red-Black Tree model implementation, pseudo-code display support, controller connection support, debugging and testing, main report and slide preparation, and partial controller/view development.
Nguyen Xuan Thuy	202417202	Partial View and Controller development, exception handling implementation, and input error handling support.

3 Technical Foundation

3.1 Technology Stack

The project is a Maven-based JavaFX desktop application. The main technologies and tools used in the project are summarized in Table 2.

Technology	Usage in project
Java	Main programming language. The Maven compiler plugin is configured with source and target version 23.
JavaFX 21.0.6	Used for GUI controls, FXML loading, Canvas drawing, dialogs, sliders and animation support.
Maven	Used for dependency management, project compilation and JavaFX run configuration.
FXML	Used to define the layout of the main menu, tree selection screen, workspace, settings screen and credit screen.
CSS	Used for base styling, dark-mode styling and visual consistency across the application.
Git and GitHub	Used for version control, source code management and team collaboration.

Table 2: *Main technologies used in the project*

3.2 User Interface Framework and Screens

The user interface is built with JavaFX. The application starts from `com.Launcher`, sets the stage title to `Tree Operation`, loads the application icon and delegates screen navigation to `NavigationManager`.

The UI is divided into the following screens:

- `main-menu-view.fxml`: entry screen with navigation to start, settings and help/credits.
- `tree-selection-view.fxml`: tree type selection screen.
- `workspace.fxml`: main working screen for tree operations and visualization.
- `settings-view.fxml`: resolution and theme configuration screen.
- `credit-view.fxml`: acknowledgements, member list and image-based user instructions.

3.3 Supported Tree Algorithms

The project supports five tree types through `TreeType` and `TreeFactory`:

- **General Tree**: supports a node with multiple children.
- **Binary Tree**: supports parent-based insertion with left and right child constraints.
- **Binary Search Tree**: supports ordered insertion, search, deletion and traversal.
- **AVL Tree**: supports self-balancing with height updates, balance factor checking and rotations.
- **Red-Black Tree**: supports color-based balancing, recoloring, transplant operations and fix-up steps.

The common operations are declared in `AbstractTree`:

```

1 public abstract void create(int value);
2 public abstract boolean insert(int parentValue, int value);
3 public abstract boolean delete(int value);
4 public abstract boolean update(int currentValue, int newValue);
5 public abstract List<Integer> traverse(TraversalType type);

```

```
6 public abstract boolean search(int value);  
7 public abstract int getHeight();  
8 public abstract int getNumberOfNodes();
```

4 Detailed Design

4.1 Use Case Diagram

This use case diagram gives an overview of how the user interacts with the **Tree Operation** application. It focuses on the main actor of the system and the general functions that the application provides.

4.1.1 System Actor

The main actor interacting with the system is the **User**. The User operates the program through the graphical user interface by clicking buttons, entering input values and selecting available functions.

4.1.2 General Functions

The main functions of the system include selecting a tree type, creating a tree, inserting nodes, deleting nodes, updating node values, searching for a value and traversing the tree. The application also supports visualization-related functions such as playing animations, pausing or resuming the animation, adjusting speed, zooming or panning the canvas, and using undo or redo to review previous tree states. In addition, the user can open the help page, view basic instructions and change visual settings such as theme or resolution.

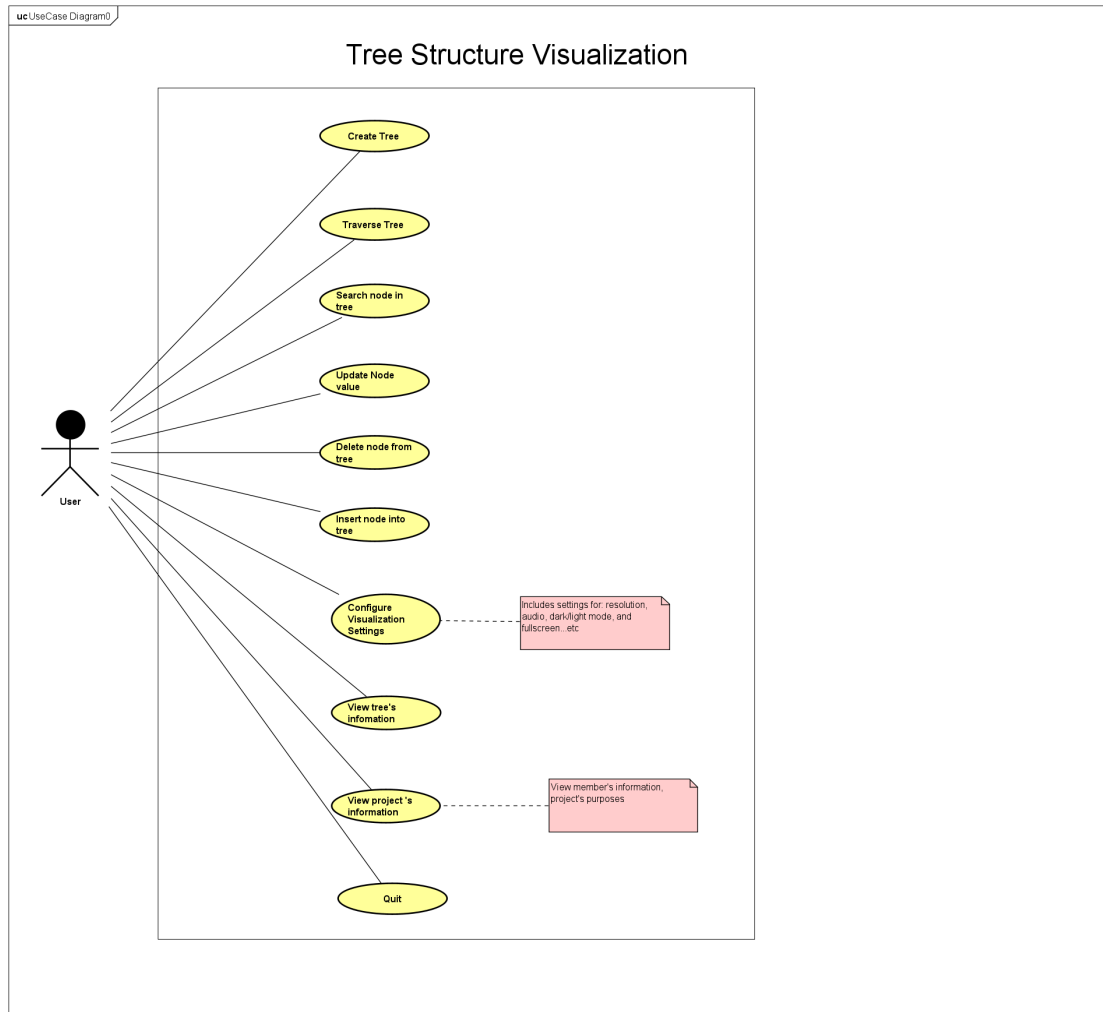


Figure 1: Use case diagram

4.2 Class Diagrams

This section presents the class design of the **Tree Operation** application. The class diagrams are organized into two levels. The first level gives a general view of the whole system and explains the main components. The second level presents detailed diagrams for the core MVC packages and supporting packages used by the application.

4.2.1 General Class Diagram

Overview

The general class diagram provides an overall view of the application architecture. It shows that the system is built around the Model-View-Controller structure. The **Model** package stores tree data and implements tree algorithms. The **View** package contains visual objects, layout logic, rendering classes, animation classes and pseudo-code display components. The **Controller** package connects user actions from JavaFX screens to the model and visualization logic. In addition, the **Theme** and **Util** packages provide supporting services for visual styling, input validation, alert dialogs and reusable helper methods.

At a high level, the user works with the application through the graphical interface. Controller classes

receive button events, validate input and call the corresponding tree operations. The model then updates the logical tree and emits operation steps. These steps are received by the visualization controller and converted into animations on the canvas. Theme-related classes keep the visual appearance consistent between light mode and dark mode, while utility classes reduce duplicated code in validation, alert handling and visual tree lookup.

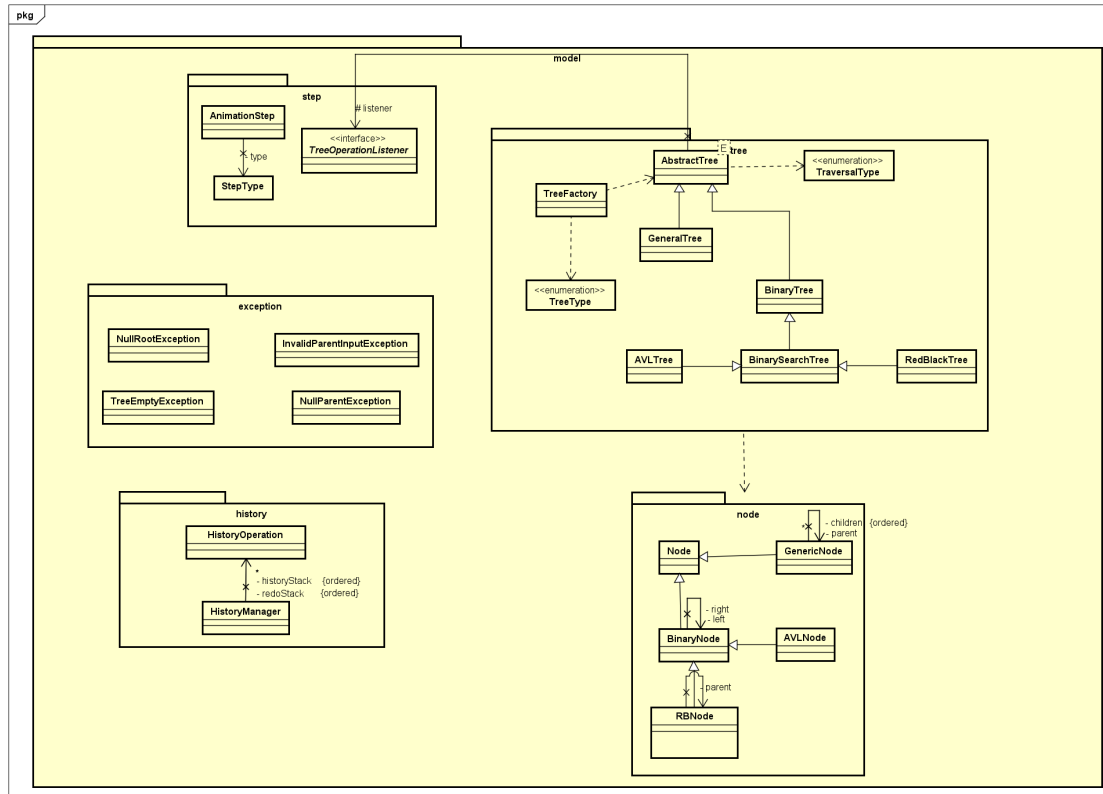


Figure 2: General class diagram

Component Analysis

The main components in the general class diagram are summarized in Table 3.

Component	Main classes/packages	Responsibility
Model	AbstractTree, GeneralTree, BinaryTree, BinarySearchTree, AVLTree, RedBlackTree, node classes	Stores the logical tree structure and implements algorithms for creating, inserting, deleting, updating, searching and traversing trees. This component is independent from JavaFX drawing code.
Step and History	StepType, AnimationStep, TreeOperationListener, HistoryManager, HistoryOperation	Defines the communication between tree algorithms and visualization. It records important operation steps and supports undo/redo by storing previous operations or tree states.
View and Visualization	VisualTree, VisualNode, VisualEdge, TreeCanvas, renderers, layout classes, pseudo-code display	Represents the tree visually. It converts logical nodes into drawable objects, calculates node positions, renders nodes and edges, and displays pseudo-code or explanation messages.
Animation	AnimationManager, StepAnimationStrategy, SearchStepStrategy, StructureStepStrategy, BalanceStepStrategy, RedBlackStepStrategy	Converts recorded algorithm steps into visual effects such as highlighting, inserting, deleting, moving, rotating and recoloring nodes.
Controller	MainMenuController, TreeSelectionController, WorkspaceController, TreeVisualizationController, NavigationManager	Handles user interaction, screen navigation, tree selection, input validation, operation execution and animation playback. It acts as the bridge between the interface, the model and the view.
Theme	Theme, LightTheme, DarkTheme, ThemeManager, ThemeType	Manages the visual style of the application. It provides colors, stylesheet paths and theme switching logic so that screens, canvas elements and dialogs can follow a consistent light or dark appearance.
Utility	InputValidator, AlertUtils, VisualTreeUtils	Provides reusable helper methods for parsing integer input, showing error alerts, finding visual nodes or edges and extracting values needed by the visualization logic.

Table 3: Component analysis of the general class diagram

4.2.2 Detailed Class Diagram

The detailed class diagrams show the internal structure of each main part of the system. The core design is explained through the Model, View and Controller packages. The Theme and Util packages are also included because they support the interface and reduce duplicated code across controllers and visualization classes.

Model Package

The model package contains the logical data structures and algorithm-related classes of the application. The central class is `AbstractTree`, which defines the common operations for all tree types, including `create()`, `insert()`, `delete()`, `update()`, `search()`, `traverse()`, `getHeight()` and `getNumberOfNodes()`. Concrete tree classes such as `GeneralTree`, `BinaryTree`, `BinarySearchTree`,

AVLTree and **RedBlackTree** extend or specialize this abstraction according to their own rules.

The node hierarchy is also part of the model package. **Node** stores the common value of a node, while **GenericNode**, **BinaryNode**, **AVLNode** and **RNode** add the extra information required by each tree type. For example, **GenericNode** stores a list of children, **BinaryNode** stores left and right children, **AVLNode** stores height information, and **RNode** stores parent and color information.

The model diagram also includes step, history and exception classes. **StepType**, **AnimationStep** and **TreeOperationListener** allow tree algorithms to report meaningful events to the visualization layer. **HistoryManager** and **HistoryOperation** support operation history and undo/redo behavior. Custom exception classes are used to represent invalid tree states or invalid parent-child relationships.

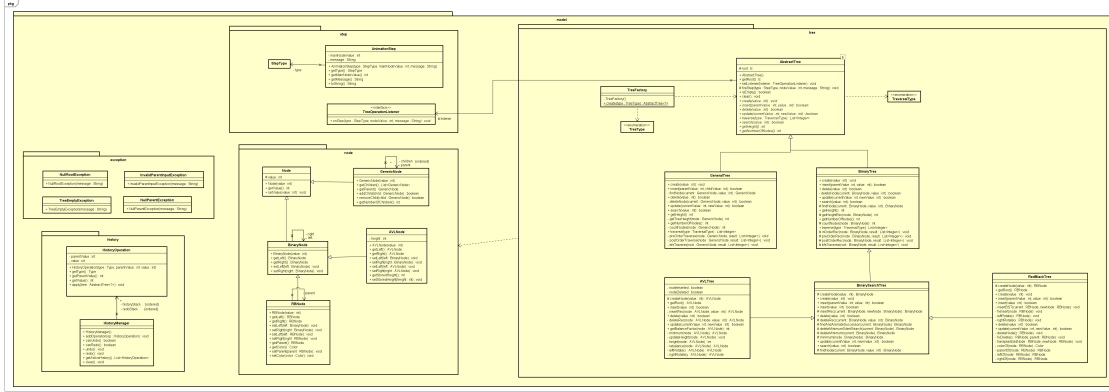


Figure 3: Model package class diagram

View Package

The view package contains the classes responsible for displaying and animating the tree. The visual model is represented by **VisualTree**, **VisualNode** and **VisualEdge**. These classes do not implement tree algorithms; instead, they store drawing-related information such as position, opacity, radius, edge progress and highlight state.

VisualTreeMapper converts the logical tree from the model package into visual objects that can be drawn on the canvas. Layout classes such as **GeneralTreeLayout** and **BinaryTreeLayout** calculate suitable node positions for different tree structures. **TreeCanvas**, camera classes and renderer classes are responsible for drawing nodes, edges and labels while supporting zoom and pan operations.

The view package also contains the animation system. **AnimationManager** controls the playback of animation objects, while different animation strategies convert operation steps into suitable visual effects. For example, search-related steps can highlight visited nodes, structure-related steps can show insertion or deletion, AVL steps can show rotations, and Red-Black steps can show recoloring. This design makes the visualization flexible because new animation behavior can be added without changing the tree algorithms.

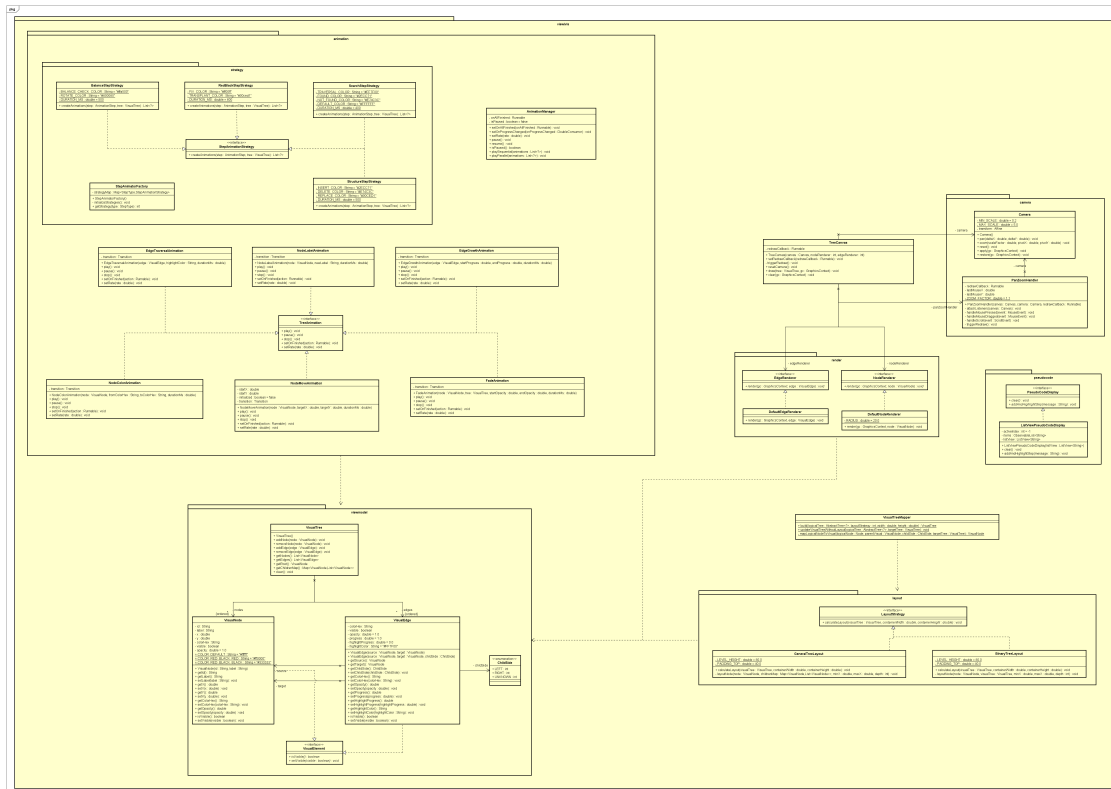


Figure 4: View package class diagram

Controller Package

The controller package connects the JavaFX interface with the model and view layers. Controllers for the main menu, settings page, credit page and tree selection page handle screen-specific user actions. **NavigationManager** centralizes screen switching and keeps navigation logic consistent across the application.

WorkspaceController is the main controller for tree operations. It reads input values from the interface, validates them, calls the selected tree operation and updates displayed information such as root value, node count, height, traversal result and operation status. It also handles workspace features such as undo, redo, animation controls and operation buttons.

TreeVisualizationController is responsible for connecting algorithm steps to the visual animation system. It implements the listener/animator role, receives steps emitted by the model, stores them as animation steps and plays them on the canvas. This separation allows **WorkspaceController** to focus on user operations while **TreeVisualizationController** focuses on drawing and animation behavior.

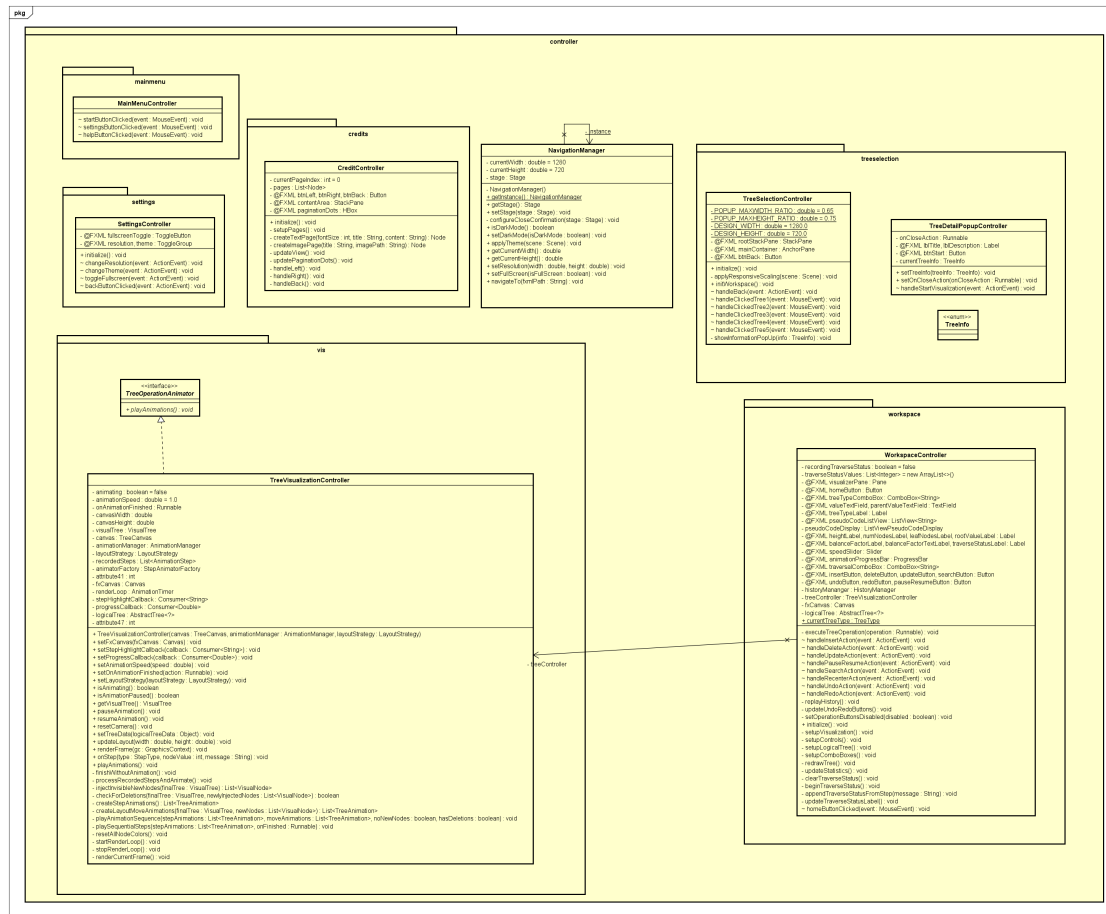


Figure 5: *Controller package class diagram*

Theme Package

The theme package supports the visual appearance of the application. It is not part of the tree algorithm itself, but it is important for keeping the user interface consistent. The **Theme** interface defines common theme-related methods such as checking dark mode, getting the stylesheet path and providing colors for canvas edges, node fill, node stroke, node text and credit page elements.

`LightTheme` and `DarkTheme` implement the `Theme` interface with different color rules. This allows the application to change appearance without changing the controller or drawing logic. Instead of hard-coding colors in many classes, the UI can ask the current theme for the correct color.

ThemeManager controls the active theme and stores the current **ThemeType**. It is implemented as a manager class that provides access to the current theme, applies the selected theme to JavaFX scenes and dialogs, and adjusts images when necessary. Therefore, this package should be placed after the View and Controller explanations as a supporting UI package.

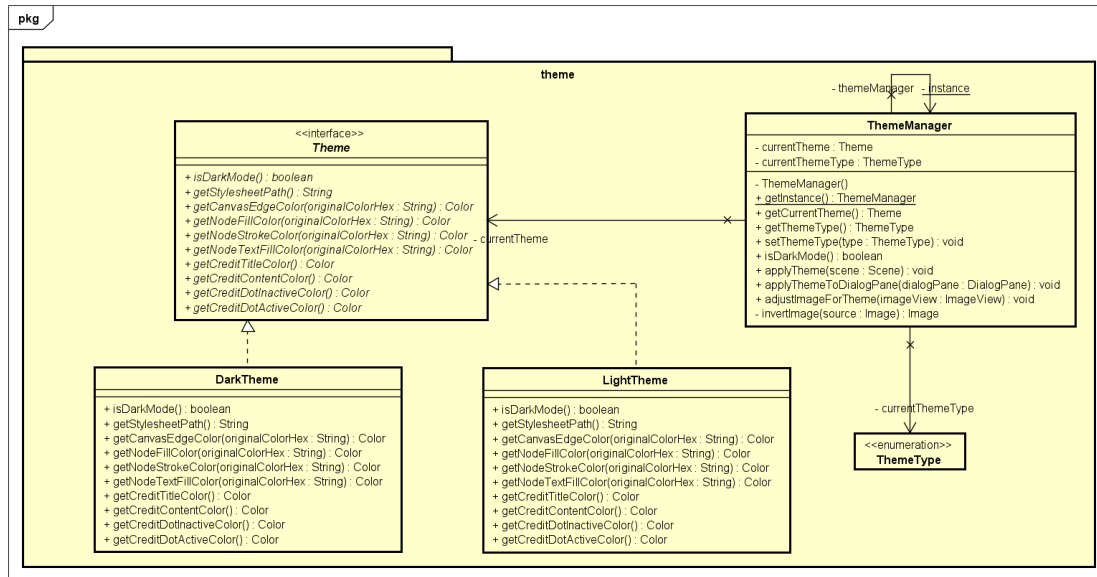


Figure 6: Theme package class diagram

Util Package

The util package contains helper classes that are used by different parts of the application. **InputValidator** validates and converts text field input into integer values before the controller sends data to the model. This helps prevent invalid input from breaking tree operations.

AlertUtils provides a reusable method for showing error alerts. Instead of writing alert creation code repeatedly in many controllers, the application can call this utility class whenever it needs to display a user-friendly error message.

VisualTreeUtils supports visualization-related lookup operations. It can find a visual node by value or id, find incoming or outgoing visual edges, parse replacement values from messages and get the logical node color. These helper methods make the visualization controller and animation strategies shorter and easier to maintain. Therefore, the util package fits best as a supporting package after the core MVC class diagrams.

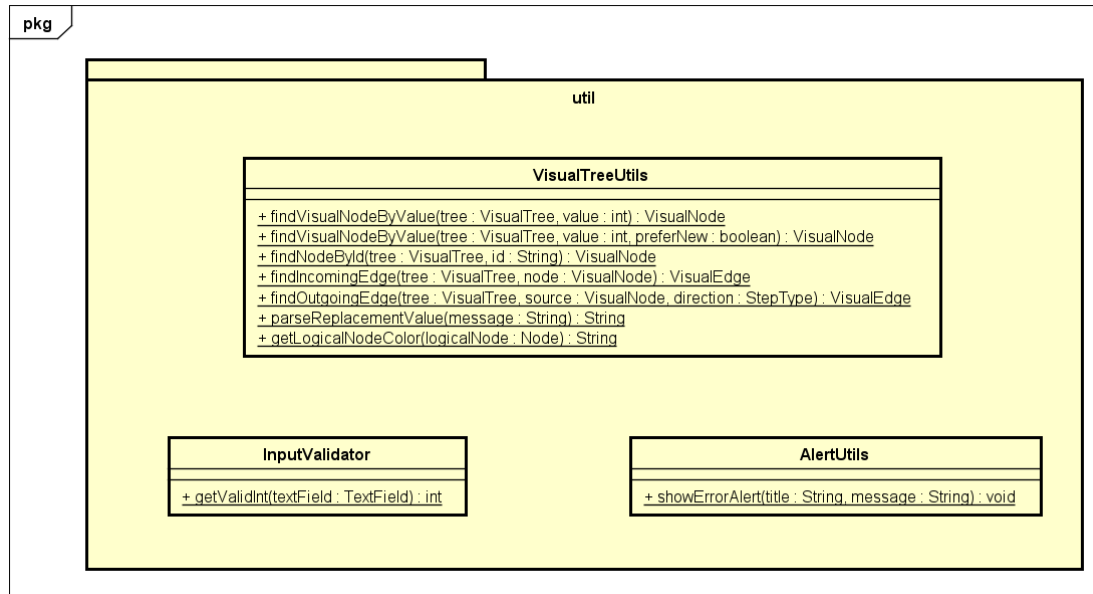


Figure 7: Util package class diagram

4.3 Important Runtime Flow

The following flow describes an insertion operation in the workspace:

1. The user enters a value and clicks the Insert button.
2. `WorkspaceController.handleInsertAction()` validates input through `InputValidator`.
3. The controller calls `logicalTree.create()` or `logicalTree.insert()`.
4. The concrete tree performs its algorithm and calls `fireStep()` for important events.
5. `TreeVisualizationController.onStep()` receives each step and stores it as an `AnimationStep`.
6. After the model operation finishes, `playAnimations()` converts recorded steps to concrete animations.
7. The canvas is redrawn until all animations finish.
8. Statistics such as height, node count, root value and AVL balance factor are updated.

4.4 Operation Step Model

The project uses a listener-based design to separate algorithm logic from UI behavior.

```

1  protected void fireStep(StepType type, int nodeValue, String message) {
2      if (listener != null) {
3          listener.onStep(type, nodeValue, message);
4      }
5  }
  
```

The `StepType` enumeration acts as a shared vocabulary between the model and visualization controller. It includes search steps, structural steps, AVL balancing steps, Red-Black tree steps and General Tree steps.

Step group	Examples
Search and traversal	VISIT, COMPARE, GO_LEFT, GO_RIGHT, GO_CHILD, ADD_TO_RESULT
Structural changes	INSERT_NODE, DELETE_NODE, REPLACE_VALUE, ADD_CHILD, REMOVE_CHILD
AVL balancing	UPDATE_HEIGHT, CHECK_BALANCE, ROTATE_LEFT, ROTATE_RIGHT
Red-Black balancing	RECOLOR, FIX_START, TRANSPLANT
Result state	FOUND, NOT_FOUND, DONE

Table 4: *Main operation step groups*

4.5 Visualization and Animation Design

`TreeVisualizationController` implements both `TreeOperationAnimator` and `TreeOperationListener`. This lets it listen to model steps and later play them as animations.

The animation system uses a factory to map each step type to a strategy:

```

1 strategyMap.put(StepType.VISIT, searchStrategy);
2 strategyMap.put(StepType.INSERT_NODE, structureStrategy);
3 strategyMap.put(StepType.ROTATE_LEFT, balanceStrategy);
4 strategyMap.put(StepType.RECOLOR, rbStrategy);

```

This design keeps each animation category focused:

- `SearchStepStrategy`: highlights search and traversal movement.
- `StructureStepStrategy`: handles inserting, deleting and replacing values.
- `BalanceStepStrategy`: handles AVL balance checks and rotations.
- `RedBlackStepStrategy`: handles recoloring and Red-Black fix-up events.

4.6 Layout Design

The project defines a `LayoutStrategy` interface:

```

1 public interface LayoutStrategy {
2     void calculateLayout(VisualTree visualTree,
3                          double containerWidth,
4                          double containerHeight);
5 }

```

`BinaryTreeLayout` recursively divides the horizontal area into left and right subregions. `GeneralTreeLayout` divides the parent region into equal sections based on the number of children. This makes the visualization reusable for both binary and n-ary tree structures.

5 Object-Oriented Design and Techniques

This section explains how the project applies the main principles of object-oriented programming. The application is not only implemented as a collection of tree algorithms, but also organized into clear model, view and controller components. This design improves code reuse, maintainability and extensibility.

5.1 Encapsulation

Encapsulation is used to protect the internal state of tree models, visual objects and controllers. Important data such as tree roots, child pointers, node height, Red-Black colors and operation history are not accessed freely from outside classes. Instead, they are controlled through public methods.

For example, `AbstractTree` stores the root node and listener as protected fields:

```
1 public abstract class AbstractTree<N extends Node> {
2     protected N root;
3     protected TreeOperationListener listener = null;
4
5     public N getRoot() {
6         return this.root;
7     }
8
9     public void setListener(TreeOperationListener listener) {
10         this.listener = listener;
11     }
12 }
```

The root is protected so that subclasses such as `BinaryTree`, `AVLTree` and `RedBlackTree` can update it internally. However, external classes cannot directly change the root. They must use public operations such as `create()`, `insert()`, `delete()` or `clear()`.

Encapsulation is also applied in node classes:

```
1 public class BinaryNode extends Node {
2     private BinaryNode left;
3     private BinaryNode right;
4
5     public BinaryNode getLeft() { return this.left; }
6     public BinaryNode getRight() { return this.right; }
7     public void setLeft(BinaryNode left) { this.left = left; }
8     public void setRight(BinaryNode right) { this.right = right; }
9 }
```

This prevents other classes from modifying child nodes directly. In the view layer, visual objects such as `VisualNode` and `VisualEdge` also hide their drawing state. For example, opacity and progress values are clamped to valid ranges before being assigned.

```
1 public void setOpacity(double opacity) {
2     this.opacity = Math.max(0.0, Math.min(1.0, opacity));
3 }
```

In the controller layer, JavaFX components such as text fields, labels, sliders and progress bars are declared as private FXML fields. This keeps UI details inside their corresponding controller and avoids unnecessary dependencies between screens.

5.2 Inheritance

Inheritance is used mainly in the model layer to represent different kinds of trees and nodes. The tree hierarchy starts from the abstract class `AbstractTree`, which defines the common operations for all tree types.

```
1 public abstract class AbstractTree<N extends Node> { ... }
2
3 public class BinaryTree extends AbstractTree<BinaryNode> { ... }
4
5 public class BinarySearchTree extends BinaryTree { ... }
6
7 public class AVLTree extends BinarySearchTree { ... }
8
9 public class RedBlackTree extends BinarySearchTree { ... }
10
11 public class GeneralTree extends AbstractTree<GenericNode> { ... }
```

This structure allows common behavior to be reused. For example, `AVLTree` and `RedBlackTree` are both based on the idea of ordered binary search trees, but each class adds its own balancing mechanism.

Node classes also follow an inheritance structure:

- `Node`: stores the common integer value.
- `BinaryNode`: adds left and right child references.
- `AVLNode`: adds height information for AVL balancing.
- `RBNode`: adds parent reference and Red/Black color.
- `GenericNode`: supports multiple children for General Tree.

For example, `AVLNode` extends `BinaryNode` and adds height information:

```
1 public class AVLNode extends BinaryNode {
2     private int height;
3
4     public int getStoredHeight() {
5         return this.height;
6     }
7
8     public void setStoredHeight(int height) {
9         this.height = height;
10    }
11 }
```

This design avoids rewriting common node data while still allowing each tree type to have the extra information it needs.

5.3 Polymorphism

Polymorphism allows the controller to work with different tree types through the same abstract reference. In the workspace, the selected tree is stored as an `AbstractTree<?>` object:

```
1 private AbstractTree<?> logicalTree;
2
3 private void setupLogicalTree() {
4     logicalTree = TreeFactory.create(currentTreeType);
5 }
```

```

5     logicalTree.setListener(treeController);
6     treeController.setTreeData(logicalTree);
7 }

```

At runtime, `TreeFactory` can return a `GeneralTree`, `BinaryTree`, `BinarySearchTree`, `AVLTree` or `RedBlackTree`. The controller can still call the same methods:

```

1 logicalTree.insert(parentValue, value);
2 logicalTree.delete(value);
3 logicalTree.update(currentValue, newValue);
4 logicalTree.search(value);

```

The actual method that runs depends on the real tree object. For example, AVL insertion performs height updates and rotations, while Red-Black Tree insertion performs recoloring and fix-up steps.

Polymorphism also appears in the visualization layer. Different layout algorithms implement the same `LayoutStrategy` interface:

```

1 LayoutStrategy layoutStrategy =
2     (currentTreeType == TreeType.GENERAL)
3         ? new GeneralTreeLayout()
4         : new BinaryTreeLayout();

```

The controller only calls:

```

1 layoutStrategy.calculateLayout(visualTree, width, height);

```

It does not need to know whether the actual layout is for a binary tree or a general tree.

5.4 Abstraction

Abstraction is used to hide implementation details and expose only necessary operations. The most important abstraction in the model layer is `AbstractTree`:

```

1 public abstract void create(int value);
2 public abstract boolean insert(int parentValue, int value);
3 public abstract boolean delete(int value);
4 public abstract boolean update(int currentValue, int newValue);
5 public abstract List<Integer> traverse(TraversalType type);
6 public abstract boolean search(int value);
7 public abstract int getHeight();
8 public abstract int getNumberOfNodes();

```

The controller only knows that every tree supports these operations. It does not need to know how each tree performs insertion, deletion, traversal or balancing.

The project also uses interface-based abstraction. For example, `TreeOperationListener` abstracts the communication between tree algorithms and visualization:

```

1 public interface TreeOperationListener {
2     void onStep(StepType type, int nodeValue, String message);
3 }

```

Tree models only emit operation steps. They do not depend on JavaFX or drawing logic. This keeps the model layer independent from the user interface.

In the animation system, `StepAnimationStrategy` hides how each algorithm step is converted into visual effects:

```
1 public interface StepAnimationStrategy {
2     List<TreeAnimation> createAnimations(
3         AnimationStep step, VisualTree tree);
4 }
```

Different strategies handle search highlighting, structural changes, AVL rotations and Red-Black recoloring.

5.5 Association, Aggregation and Composition

The project uses several object relationships to organize its components.

5.5.1 Association

Association is a loose relationship where one object uses another object. For example, `AbstractTree` is associated with `TreeOperationListener`. The tree can work without a listener, but when a listener is attached, it sends operation steps to the visualization controller.

```
1 protected void fireStep(StepType type, int nodeValue, String message) {
2     if (listener != null) {
3         listener.onStep(type, nodeValue, message);
4     }
5 }
```

5.5.2 Aggregation

Aggregation is a weak “has-a” relationship. `TreeVisualizationController` aggregates helper objects such as `TreeCanvas`, `AnimationManager`, `LayoutStrategy` and `StepAnimatorFactory`.

```
1 private VisualTree visualTree;
2 private TreeCanvas canvas;
3 private AnimationManager animationManager;
4 private LayoutStrategy layoutStrategy;
5 private StepAnimatorFactory animatorFactory;
```

These objects cooperate to display and animate the tree, but they are still meaningful as separate components.

5.5.3 Composition

Composition is a strong “part-of” relationship. For example, `VisualTree` owns its visual nodes and edges:

```
1 public class VisualTree {
2     private List<VisualNode> nodes;
3     private List<VisualEdge> edges;
4
5     public VisualTree() {
6         this.nodes = new ArrayList<>();
```

```
7         this.edges = new ArrayList<>();
8     }
9 }
```

Similarly, `HistoryManager` owns its undo and redo stacks. These internal structures belong to the manager and represent its state.

5.6 Design Patterns

5.6.1 Model-View-Controller Pattern

The whole application follows the MVC pattern.

- **Model:** stores data, implements tree algorithms and emits operation steps.
- **View:** contains FXML files, CSS files, visual nodes, visual edges, renderers and canvas drawing components.
- **Controller:** handles user actions, calls model operations and updates the visualization.

This separation makes the system easier to maintain. Tree algorithms do not depend on JavaFX controls, and the view does not directly perform algorithm logic.

5.6.2 Factory Pattern

`TreeFactory` creates the correct concrete tree based on `TreeType`. This avoids repeating tree creation logic in multiple controllers.

```
1 public static AbstractTree<?> create(TreeType type) {
2     switch (type) {
3         case GENERAL: return new GeneralTree();
4         case BINARY: return new BinaryTree();
5         case BINARY_SEARCH: return new BinarySearchTree();
6         case AVL: return new AVLTree();
7         case RED_BLACK: return new RedBlackTree();
8         default: throw new IllegalArgumentException(...);
9     }
10 }
```

5.6.3 Strategy Pattern

The strategy pattern is used for layout and animation behavior.

- **LayoutStrategy:** implemented by `BinaryTreeLayout` and `GeneralTreeLayout`.
- **StepAnimationStrategy:** implemented by search, structure, balance and Red-Black animation strategies.

This design allows the application to select different algorithms without changing the controller workflow.

5.6.4 Singleton Pattern

`NavigationManager` and `ThemeManager` are implemented as singleton managers. This ensures that the application has one consistent navigation manager and one active theme manager.

```

1  private static NavigationManager instance;
2
3  public static NavigationManager getInstance() {
4      if (instance == null) {
5          instance = new NavigationManager();
6      }
7      return instance;
8  }

```

5.7 Exception Handling

Exception handling is used to keep the application stable when invalid input or invalid operations occur. User input is validated before being passed to tree operations:

```

1  public static int getValidInt(TextField textField)
2      throws NullPointerException, NumberFormatException {
3      if (textField == null)
4          throw new NullPointerException("TextField is null");
5
6      String valueStr = textField.getText();
7      if (valueStr == null || valueStr.trim().isEmpty())
8          throw new NumberFormatException("TextField is empty");
9
10     return Integer.parseInt(valueStr.trim());
11 }

```

The workspace controller catches common errors and displays user-friendly alerts:

```

1  try {
2      operation.run();
3  } catch (NumberFormatException e) {
4      AlertUtils.showErrorAlert("Invalid Input",
5          "Please enter a valid integer value.");
6      return;
7  } catch (IllegalArgumentException | IllegalStateException
8      | UnsupportedOperationException e) {
9      AlertUtils.showErrorAlert("Operation Error", e.getMessage());
10     return;
11 } catch (Exception e) {
12     AlertUtils.showErrorAlert("Unexpected Error",
13         "An unexpected error occurred: " + e.getMessage());
14     return;
15 }

```

Some expected operation results, such as duplicate insertion or unsuccessful search, are handled by boolean return values and algorithm steps instead of exceptions. This makes the system easier to control and prevents normal user mistakes from crashing the application.

6 User Guide

6.1 System Requirements

- Operating system: Windows, Linux or macOS with JavaFX support.
- JDK: Java 23 or newer is recommended because the Maven compiler is configured for source/-target 23.
- Maven: Maven can be used directly, or the Maven wrapper scripts `mvnw` and `mvnw.cmd` can be used.

6.2 Installation

6.2.1 Building from Source

From the project root, move into the `app` directory and run:

```
1 cd app
2 ./mvnw clean javafx:run
```

On Windows PowerShell:

```
1 cd app
2 .\mvnw.cmd clean javafx:run
```

6.2.2 Project Entry Point

The main class is:

```
1 com.Launcher
```

It initializes the JavaFX stage, applies the application icon and navigates to the main menu FXML screen.

6.3 Using the Application

6.3.1 Initialization

When the application starts, users enter the main menu. From there, they can start the tree visualization workflow, open settings or read the help/credit page. The tree selection screen lets users choose one of the supported tree types.

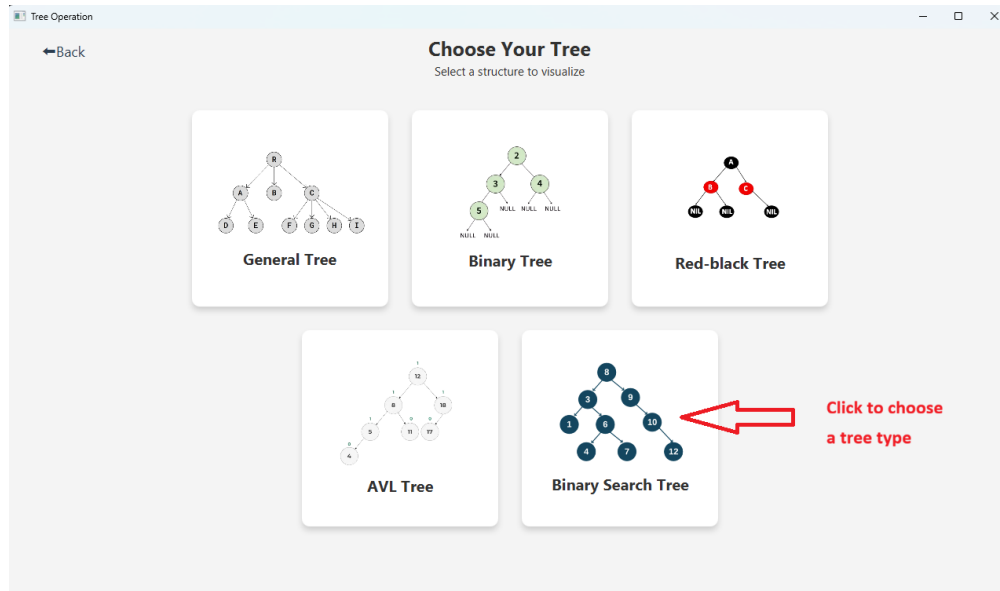


Figure 8: Selecting a tree type

6.3.2 Executing Operations

In the workspace, users can input integer values and execute:

- **Insert:** create the first node or add a new node according to the selected tree rules.
- **Delete:** remove an existing node and visualize replacement/fix-up steps.
- **Update:** replace a node value using a dialog.
- **Search:** highlight the path used to find a value.
- **Traverse:** display the selected traversal order in the status label.

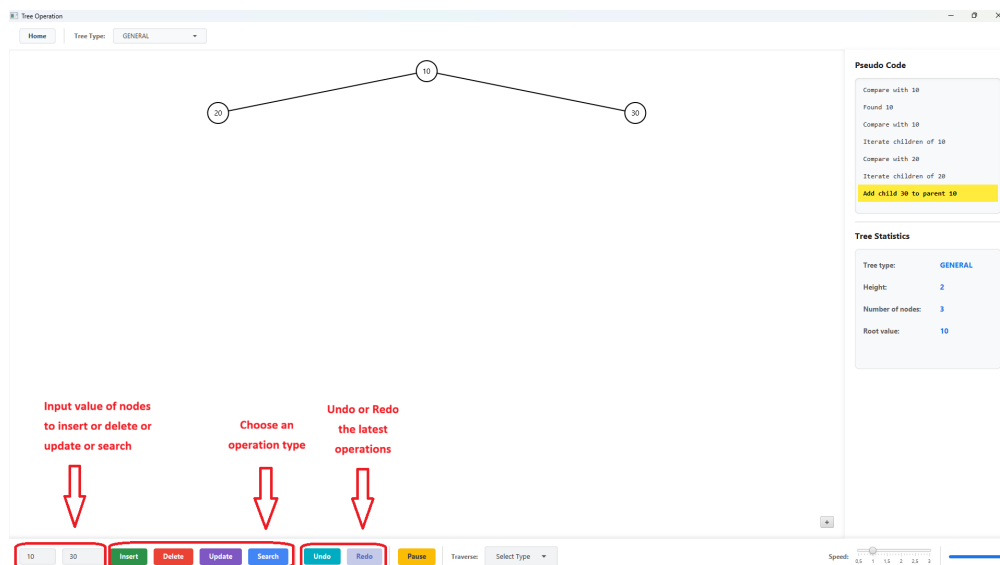


Figure 9: Inserting a node

6.3.3 Animation Control

Users can control animation playback with:

- Pause/Play button.
- Speed slider.
- Progress bar.
- Recenter/camera reset.
- Pan and zoom interaction on the canvas.

6.3.4 Visual Aids

The workspace provides several learning aids:

- Algorithm step messages displayed in the pseudo-code list.
- Highlighted nodes and edges during operation playback.
- Tree statistics: height, number of nodes, root value and AVL balance factor.
- Traversal result display.
- Undo/redo by replaying operation history.

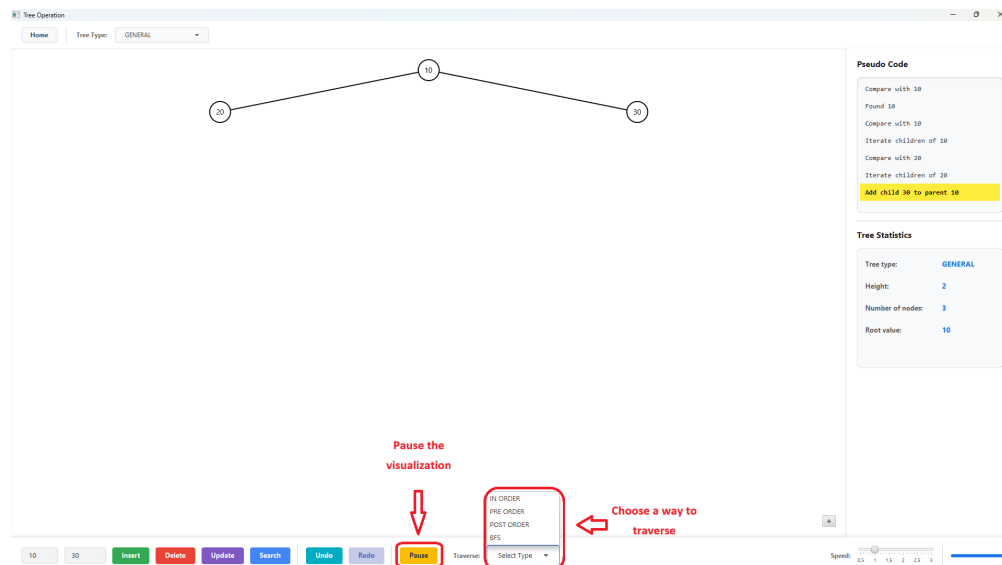


Figure 10: Traversal workflow

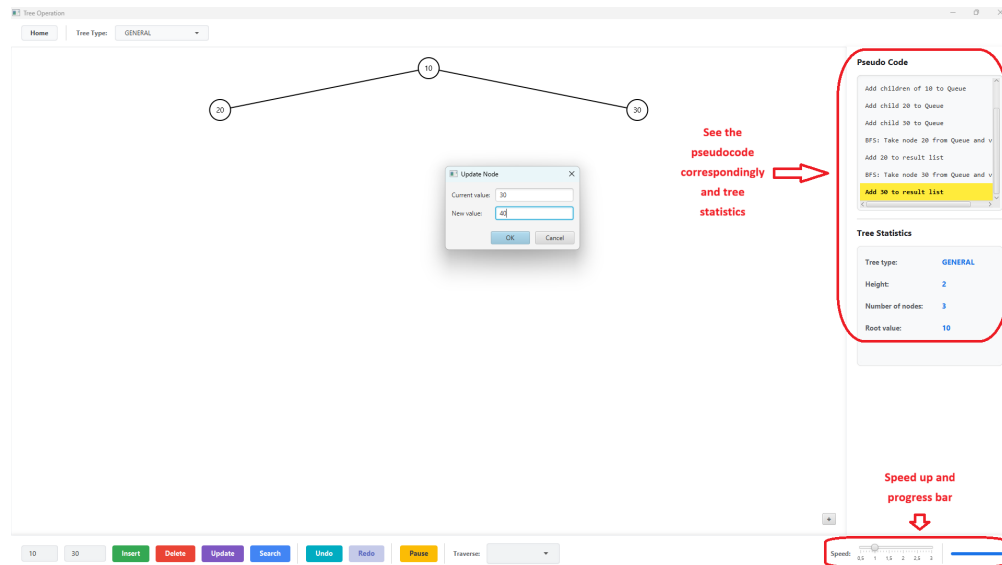


Figure 11: *Updating a node*

6.4 Video Demonstration

The repository does not currently contain a video demonstration link. The recommended demonstration should show:

- Starting the application from Maven.
- Selecting each supported tree type.
- Insert, delete, update, search and traversal operations.
- AVL rotations and Red-Black recoloring/fix-up.
- Pause/resume, speed control, dark mode and undo/redo.

7 Conclusion

7.1 Key Achievements

The project successfully implements an interactive tree data structure visualization application with a clear object-oriented architecture. It supports five tree types, common tree operations, step-based algorithm messages, animated visualization, JavaFX screens, theme management and operation history.

The most important achievement is the separation between model logic and visual behavior. Tree algorithms emit semantic steps, while the visualization layer decides how those steps should appear on screen. This design makes the model easier to test and the animation layer easier to extend.

7.2 Current Limitations

The current implementation still has several limitations:

- Input is limited to integer values.
- The application does not persist trees between sessions.

- The report source does not include a confirmed final demo video link.
- Automated tests are configured through JUnit dependencies, but the main app module can still benefit from more dedicated unit tests for every tree operation and edge case.

7.3 Future Enhancements

Future work can include:

- Saving and loading tree states from files.
- Exporting animation steps or screenshots.
- Adding more tree structures such as B-Tree, Heap, Trie or Segment Tree.
- Adding richer pseudo-code synchronization with exact algorithm line numbers.
- Adding complete automated tests for model invariants and controller workflows.

7.4 Learning Outcomes

Through this project, the team practiced:

- Applying OOP principles in a real Java project.
- Designing reusable model, view and controller components.
- Implementing and visualizing complex tree algorithms.
- Using JavaFX Canvas, FXML, CSS and Maven.
- Applying design patterns such as MVC, Factory, Observer, Strategy and Singleton.